Lab 4. Using Streams

Authors: Yao Zhao, Yida Tao

1. Creating streams

Streams can be created from various data sources, especially collections. `StreamCreation.java` shows a lot of examples how Streams be created.

2. Intermediate operations (Laziness, Stateful/Stateless)

An important characteristic of intermediate operations is laziness. Look at this sample where a terminal operation is missing:

```
Stream.of("CS", "209", "A").filter(s -> {
    System.out.println("filter: " + s);
    return true;
});
```

When executing this code snippet, nothing is printed to the console. That is because intermediate operations are lazy and will only be executed when a terminal operation is present.

```
Stream.of("209", "CS", "303", "A", "B")
        .sorted((s1, s2) -> {
            System.out.printf("sort: %s; %s\n", s1, s2);
            return s1.compareTo(s2);
        })
        .filter(s -> {
            System.out.println("filter: " + s);
            return s.startsWith("A");
        })
        .map(s -> {
            System.out.println("map: " + s);
            return s.toLowerCase();
        })
        .forEach(s -> System.out.println("forEach: " + s));
```

Observe the output above. The intermediate operation `sorted` is a *stateful* operation, which needs to see all elements before it can produce any output. Meanwhile, `filter` and `map` are *stateless* operations, which can process each element independently. Try to change the order of these intermediate operations and see how the output changes.

For more examples, please refer to `StreamProcessingOrder.java`.

3. Reusing Streams

Streams cannot be reused. As soon as you call any terminal operation the stream is closed:

```
Stream<String> stream = Stream.of("CS", "209", "A").filter(s ->
s.startsWith("C"));

stream.anyMatch(s -> true); // ok
stream.noneMatch(s -> true); // exception
```

Calling `noneMatch` after `anyMatch` on the same stream results in the following exception:

```
Exception in thread "main" java.lang.IllegalStateException: stream has already
been operated upon or closed
  at java.util.stream.AbstractPipeline.evaluate (AbstractPipeline.java:229)
  at java.util.stream.ReferencePipeline.noneMatch(ReferencePipeline.java:459)
  at StreamProcessingOrder.main(StreamProcessingOrder.java:95)
```

To overcome this limitation, we have to to create a new stream chain for every terminal operation we want to execute, e.g., we could create a stream supplier to construct a new stream with all intermediate operations already set up:

```
Supplier<Stream<String>> streamSupplier =
        () -> Stream.of("CS", "209", "A")
                .filter(s -> s.startsWith("C"));

streamSupplier.get().anyMatch(s -> true);   // ok
streamSupplier.get().noneMatch(s -> true);  // ok
```

Check `StreamReuse.java` for sample code.

4. Stream Reduction

The JDK contains many terminal operations (such as average, sum, min, max, and count) that return one value by combining the contents of a stream. These operations are called *reduction* operations.

The JDK also contains reduction operations that return a collection instead of a single value. Many reduction operations perform a specific task, such as finding the average of values or grouping elements into categories.

The JDK provides you with the general-purpose reduction operations `reduce` and `collect`. Please refer to this official guide for examples of these two operations.

We also provide a `StreamReduction.java` to demonstrate common usages for stream reduction.